LOGOTIPO DA UNIVERSIDADE

LOGOTIPO DO CURSO

APLICATIVO DE MOBILIDADE URBANA

TrueCompass

Alocação ótima de motoristas e rota de menor tempo sobre a malha viária real de São Paulo, com corretude demonstrada e ganho medido

Bianca Murcela Mascamudo · RA 8261104797 Carolina Chaves · RA 826182449 Carolina Yukari Kague · RA 8261110898
Gabryela Araujo · RA 826199094 Kauan Sousa · RA 8261113181 Wildiner Lopes · RA 8261102715
Orientador: Prof. Raulison Resende
Universidade São Judas Tadeu · Matemática Computacional Aplicada (0064481) · Campus _____ · 2026/2

● O PRODUTO

O aplicativo

Pedir a corrida
origem pelo GPS ou escolhida no mapa; destino por busca de endereço.

Achar quem chega antes
não o mais perto em linha reta, e sim o de menor tempo real pelas ruas.

Acompanhar ao vivo
o carro anda no mapa e o estado da corrida chega por WebSocket.

Pagar só no fim
o pagamento só existe depois de concluída — o fluxo não deixa pular etapa.

● POR QUE IMPORTA

Contexto e problema

A distância em linha reta mente.
No cenário medido, um motorista a 275 m do passageiro leva 5,8 min para chegar; outro, a 548 m, leva 1,0 min — o primeiro está do outro lado do rio.

k candidatos, k varreduras do grafo.
A alocação ingênua roda uma busca de rota completa para cada candidato. É computação jogada fora em cada pedido.

● O QUE FOI ENTREGUE

Objetivos

OBJETIVO GERAL

Formalizar a malha viária como grafo ponderado direcionado G = (V, E, w), com w em segundos, e implementar a camada algorítmica que devolve o motorista de menor tempo real de chegada e a rota ótima até o destino — com corretude demonstrada e ganho medido.

OBJETIVOS ESPECÍFICOS — ETAPAS CONCLUÍDAS

1. Malha como grafo direcionado em lista de adjacência, O(V + E); ciclo da corrida como DAG.
2. Dijkstra e A* na mesma rotina — o Dijkstra é o A* com h = 0 — com heurística provada admissível e consistente.
3. Pareamento de k motoristas com UMA busca no grafo reverso; quadtree no filtro espacial e Kahn no fluxo de estados.
4. Heurística ALT: marcos e desigualdade triangular sobre a malha real.
5. Aplicação web ponta a ponta: API FastAPI e app React com mapa e tempo real.

● COMO FOI CONSTRUÍDO

Arquitetura

Setas de dependência sempre apontando para dentro: a camada matemática não importa ninguém, e por isso roda sem banco, servidor nem rede.

frontend/ app React + TypeScript
mapa Leaflet, bússola, folha arrastável, WebSocket

↓

backend/api/ FastAPI
rotas HTTP e WebSocket; publica só DEPOIS do commit

↓

backend/dominio/ regras do negócio
DAG da corrida, alocação otimista, tarifa

↓

matematica/algoritmos/ matemática pura
grafo · A*/Dijkstra · ALT · quadtree · Kahn · matriz

Matriz de adjacência V×V mediu 1.067× mais memória: descartada.

● O MOTOR

Como uma corrida acontece

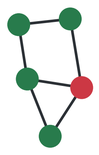
1. **Pedido** — o passageiro marca origem e destino
2. **Quadtree** — filtra os motoristas do entorno em O(log n + k)
3. **Busca reversa** — UMA busca no grafo invertido mede o tempo real de todos
4. **A* com ALT** — marcos e desigualdade triangular devolvem a rota ótima
5. **DAG da corrida** — a transição de estado é validada antes de existir
6. **Commit, e só então WebSocket** — o cliente nunca vê estado que o rollback desfez

● O MAPA

Da matriz ao grafo

.	A	.	.
.	A	S	.
#	A	.	.
.	A	>	.

→



Um caractere = uma célula. . rua A avenida S semáforo > mão única # intransponível

E[espera] = r² / 2c no peso da aresta que CHEGA ao cruzamento

● O QUE FOI MEDIDO

Resultados

Na malha real: 118.267 cruzamentos, 180.424 trechos de rua e 3.542 semáforos. Em todos os casos a rota devolvida é a mesma — o ganho é de busca, nunca de qualidade.

14,3× menos nós na rota — de Dijkstra para A* com ALT

10,7× menos nós no pareamento — de k buscas para 1 reversa

154,8× menos comparações — da varredura linear à quadtree

0 rotas subótimas — em toda a validação

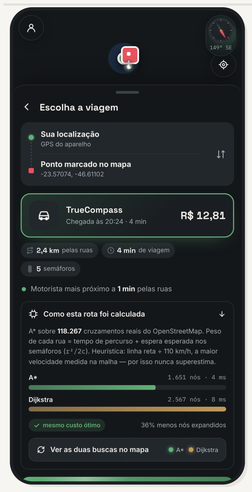
NÓS EXPANDIDOS EM 25 ROTAS ORIGEM–DESTINO

Dijkstra	1.703.955	
A* haversine	1.329.866	172 ms/rota
A* com ALT	119.398	14 ms/rota

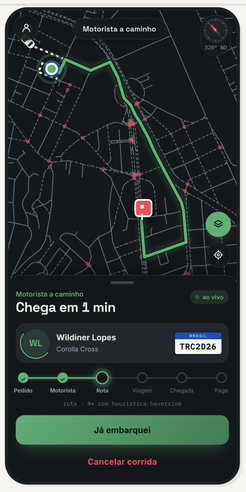
Mesmo custo ótimo nas três — conferido rota a rota contra o Dijkstra.

● DEMONSTRAÇÃO

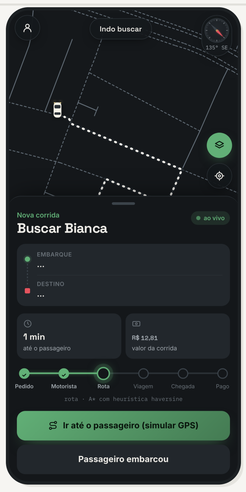
O app funcionando



A* × Dijkstra na rota pedida



motorista alocado, ao vivo



o lado do motorista

INDICADORES DE ENTREGA

- **Semáforos reais no peso das arestas**
  3.542 · 67% das rotas mudam de traçado
- **Pré-processamento do ALT**
  16 marcos · 10,3 s · artefato de 29,8 MB
- **Testes automatizados**
  183 verdes + 20 no navegador
- **Estratégia de validação**
  oráculo: Dijkstra confere o A*, varredura confere a quadtree

CONSIDERAÇÕES FINAIS

O ganho veio de trocar o algoritmo, não de comprar máquina: as mesmas rotas, com uma fração da busca. O que sustenta a troca é a demonstração — admissibilidade e consistência da heurística, dualidade do grafo reverso, aciclicidade do fluxo — e não a medição isolada. O protótipo roda ponta a ponta sobre ruas reais.

TRABALHOS FUTUROS

- Semáforos dependentes do tempo (onda verde), provando antes a propriedade FIFO.
- Algoritmo húngaro para despacho em lote, com vários pedidos simultâneos.
- Publicação: front-end no Vercel, back-end no Railway com a malha em memória.

Pilha: Python 3.11 · FastAPI · SQLAlchemy · React + TypeScript · Leaflet · WebSocket. Os algoritmos são de implementação própria — nenhuma biblioteca de grafos.

REFERÊNCIAS

CORMEN, T. H. et al. Algoritmos: teoria e prática. 4. ed. Rio de Janeiro: LTC, 2024.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, v. 4, n. 2, p. 100-107, 1968.

GOLDBERG, A. V.; HARRELSON, C. Computing the shortest path: A* search meets graph theory. In: SODA, 16., 2005, Vancouver. Proceedings. Philadelphia: SIAM, 2005. p. 156-165.

Malha: OpenStreetMap contributors (ODbL), via Overpass API.

Repositório, documentação e protótipo
github.com/Cm-Kauan/Prototipo-TrueCompass